APPENDIX A – EVIDENCE_CONTROLLER.JS

This controller module implements core digital evidence management operations including evidence listing with search and pagination, file upload with AES-256-GCM envelope encryption, secure file download with SHA-256 integrity verification, versioning, soft delete, and rollback. Every state change and data access is automatically tracked by the ISO 27037 audit logger.

```
const Evidence = require('../models/Evidence');
const Case = require('../models/Case');
const Permission = require('../models/Permission');
const EvidenceVersion = require('../models/EvidenceVersion');
const encryptionService = require('../services/encryptionService');
const storageService = require('../services/storageService');
const logAudit = require('../utils/auditLogger');
const mongoose = require('mongoose');

const checkUserAccess = async (user, evidenceId, requiredPermission =
'read') => {
  if (user.role === 'admin' || user.role === 'officer') return true;
  const evidence = await Evidence.findById(evidenceId);
  if (!evidence) return false;
  if (evidence.uploadedBy.toString() === user._id.toString()) return
true;
  const activePermission = await Permission.findOne({
    evidenceId, sharedWith: user._id, expiresAt: { $gt: new Date() }
  });
  if (!activePermission) return false;
  if (requiredPermission === 'download') {
    return activePermission.permission === 'download' ||
           activePermission.permission === 'edit_metadata';
  }
  return true;
};

const getEvidenceById = async (req, res) => {
  try {
    const evidence = await Evidence.findById(req.params.id)
      .populate('uploadedBy', 'name email role')
      .populate('caseId', 'caseNumber title status assignedOfficer');
    if (!evidence || (evidence.isDeleted && req.user.role !==
'admin')) {
      return res.status(404).json({ message: 'Evidence not found.' });
    }
    const hasAccess = await checkUserAccess(req.user, evidence._id,
'read');
    if (!hasAccess) {
      await logAudit(req, {
```

```
        action: 'VIEW_DENIED', target: evidence._id.toString(),
        details: 'Attempted to view evidence without permission',
status: 'failure'
      });
      return res.status(403).json({ message: 'Access denied:
Insufficient permissions.' });
    }
```

```
    const versions = await EvidenceVersion.find({ evidenceId:
evidence._id })
      .populate('updatedBy', 'name email role').sort({ version: -1 });
    await logAudit(req, {
      action: 'VIEW', target: evidence._id.toString(),
      details: `Viewed metadata of: "${evidence.title}"`
    });
    res.json({ evidence, versions });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

const uploadEvidence = async (req, res) => {
  try {
    if (!req.file) return res.status(400).json({ message: 'Please
upload a file.' });
    const { title, description, caseId, tags, category } = req.body;
    if (!title) return res.status(400).json({ message: 'Evidence title
is required.' });
    if (caseId) {
      const parentCase = await Case.findById(caseId);
      if (!parentCase) return res.status(404).json({ message:
'Assigned case not found.' });
    }
    const fileBuffer = req.file.buffer;
    const { encryptedBuffer, fileKeyEncrypted, fileIvHex, hash } =
encryptionService.encryptFile(fileBuffer);
    const encryptedFileURL = await
storageService.uploadFile(encryptedBuffer, req.file.originalname);
    const parsedTags = tags ? (Array.isArray(tags) ? tags :
tags.split(',').map((t) => t.trim())) : [];
    const evidence = await Evidence.create({
      title, description, caseId: caseId || undefined, uploadedBy:
req.user._id,
      encryptedFileURL, originalFilename: req.file.originalname,
      fileType: req.file.mimetype, fileSize: req.file.size, hash,
tags: parsedTags,
      category: category || 'General', fileKeyEncrypted, fileIvHex,
    });
    await logAudit(req, {
      action: 'UPLOAD', target: evidence._id.toString(),
      details: `Uploaded evidence "${title}"
(${req.file.originalname}). Hash: ${hash}`
    });
    res.status(201).json(evidence);
```

```
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

```
const downloadEvidence = async (req, res) => {
  try {
    const evidence = await Evidence.findById(req.params.id);
    if (!evidence || evidence.isDeleted) return res.status(404).json({
message: 'Evidence not found.' });
    const hasAccess = await checkUserAccess(req.user, evidence._id,
'download');
    if (!hasAccess) {
      await logAudit(req, {
        action: 'DOWNLOAD_DENIED', target: evidence._id.toString(),
        details: 'Attempted to download evidence without permission',
status: 'failure'
      });
      return res.status(403).json({ message: 'Access denied: You do
not have permissions to download this file.' });
    }
    const encryptedBuffer = await
storageService.downloadFile(evidence.encryptedFileURL);
    const decryptedBuffer =
encryptionService.decryptFile(encryptedBuffer,
evidence.fileKeyEncrypted, evidence.fileIvHex);
    const decryptedHash =
encryptionService.calculateHash(decryptedBuffer);
    if (decryptedHash !== evidence.hash) {
      await logAudit(req, {
        action: 'INTEGRITY_FAIL', target: evidence._id.toString(),
        details: 'Downloaded file hash mismatch! Integrity breached.',
status: 'failure'
      });
      return res.status(500).json({ message: 'File integrity
compromised: Hash mismatch.' });
    }
    await logAudit(req, {
      action: 'DOWNLOAD', target: evidence._id.toString(),
      details: `Downloaded evidence "${evidence.title}"`
    });
    res.setHeader('Content-Disposition', `attachment;
filename="${evidence.originalFilename}"`);
    res.setHeader('Content-Type', evidence.fileType);
    res.send(decryptedBuffer);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

module.exports = {
```

```
  checkUserAccess, getEvidenceList, getEvidenceById,
  uploadEvidence, downloadEvidence, updateEvidence,
  deleteEvidence, restoreEvidence, rollbackEvidenceVersion
};
```

Deployment URL: http://localhost:5002 (Backend API) | http://localhost:5174 (Web Dashboard)

This controller forms the operational foundation of the CivicEvidence DEMS platform. By decoupling cryptographic operations into specialized services and enforcing immutable audit logging on every request context, it ensures strict compliance with ISO/IEC 27037 and NIST SP 800-86 standards.